Midnight Calypso

Capstone Project Kickoff Document

Arizona State University | CIDSE Capstone Program | Spring 2026

Sponsor Information

Organization	Prodefense
Primary Sponsor	Matt Keeley (mkeeley@prodefense.io)
Co-Sponsor	Nathan Smith

Team Information

Name	Preferred Name	Email
Megh Kalpeshkumar Patel		mpate164@asu.edu
Parth Gupta		pgupt134@asu.edu
Nikhil Balabhadra		nbalabha@asu.edu

Project Overview

Midnight Calypso is a capstone project focused on developing tools for payload evasion that help students understand how malware evades detection by modern antivirus (AV) and endpoint detection and response (EDR) platforms. This project involves creating two separate tools: a packer/crypter for binary modification and obfuscation, and a loader generator for process injection and execution.

The packer/crypter tool will allow students to modify binaries through encryption, obfuscation, and compression techniques to evade static analysis. The loader tool will generate code that can inject and execute payloads in target processes using various injection techniques, with evasion features such as AMSI/ETW bypasses and indirect syscalls.

Cybersecurity students learn best when they can work through real attacker and defender workflows, not just read about them. This project provides hands-on experience by having students build tools that demonstrate how security controls are evaded and how malicious activity is detected. Students will gain practical knowledge of Windows internals, process injection, encryption, and the detection mechanisms used by modern security products.

Educational Objectives

- Understand PE (Portable Executable) file format internals and Windows loading mechanisms
- Learn how static analysis tools examine binaries and what patterns they detect
- Implement multiple obfuscation techniques including encryption, API hiding, and metadata manipulation
- Understand process injection techniques and Windows memory management
- Learn how EDR products detect malicious behavior and how evasion techniques work
- Gain experience with indirect syscalls and understanding the Windows syscall interface
- Develop proficiency with reverse engineering tools (IDA, Ghidra, x64dbg, PE-bear)

Project Scope

Tool 1: Packer/Crypter

The packer/crypter transforms PE executables to evade static analysis. Students will survey multiple obfuscation techniques before selecting which to implement in depth. The tool should produce standalone self-decrypting executables or packed binaries that can be loaded by the loader component.

Obfuscation Techniques to Explore

Category	Techniques	Difficulty
String Encryption	XOR, RC4, stack strings	Easy
Metadata Manipulation	Timestamp stomping, Rich header removal, section renaming	Easy
Entropy Manipulation	Base64 encoding, null padding	Easy
Junk Code Insertion	Semantic NOPs, dead code, garbage instructions	Easy-Medium
Dynamic API Resolution	GetProcAddress patterns	Medium
API Hashing	CRC32/DJB2 hash lookup	Medium
Import Table Hiding	IAT reconstruction	Medium-Hard
Control Flow Flattening	Switch-based dispatcher	Hard (Stretch)

Tool 2: Loader Generator

The loader generator creates code that can inject and execute payloads in target processes. This component focuses on process injection techniques with evasion capabilities to bypass runtime detection.

Process Injection Techniques

Technique	Description	Difficulty
Classic DLL Injection	CreateRemoteThread + LoadLibrary	Easy-Medium
Process Hollowing	CreateProcess suspended, unmap, write, resume	Medium
APC Injection	QueueUserAPC to alertable thread	Medium

Thread Hijacking	SuspendThread, SetThreadContext, ResumeThread	Medium-Hard
Module Stomping	Overwrite legitimate DLL in memory	Hard

Evasion Features

Feature	Purpose	Difficulty
AMSI Bypass	Patch AmsiScanBuffer to disable script scanning	Medium
ETW Bypass	Patch EtwEventWrite to blind telemetry	Medium
Direct Syscalls	Call NT functions directly, bypass user-mode hooks	Medium-Hard
Indirect Syscalls	JMP to syscall instruction in ntdll, evade call stack analysis	Hard
Unhooking	Restore original ntdll from disk to remove EDR hooks	Hard

Explicitly Out of Scope

- Kernel-mode drivers or rootkit components
- Code virtualization (VMProtect/Themida-style interpreters)
- Call stack spoofing
- "FUD" (fully undetectable) status as a success criterion

Project Milestones

Semester 1: Foundation and Exploration

Weeks	Focus	Deliverables
1-2	Environment Setup	Dev VMs configured, tools installed, baseline skill assessment completed
3-4	PE Format & Windows Internals	Working PE parser; team demonstrates understanding of PE structure and Windows process architecture
5-6	Basic Crypter	XOR/RC4 encryption module; self-decrypting stub that executes payload
7-8	Basic Loader	Classic DLL injection or process hollowing PoC working in isolated VM
9-12	Technique Exploration	PoC implementations for obfuscation and injection techniques; exploration reports with feasibility scores
13-14	Technique Selection	Decision matrix completed; techniques selected for full implementation in Semester 2
15-16	Showcase Prep	Demo-ready crypter and basic loader; poster and presentation materials

Semester 2: Implementation and Polish

Weeks	Focus	Deliverables
1-4	Advanced Obfuscation	Full implementation of selected obfuscation techniques (string encryption, API hiding, metadata manipulation)
5-8	Advanced Loader	Additional injection techniques; AMSI/ETW bypasses; direct and indirect syscalls integrated
9-10	Testing & Validation	Testing against Windows Defender and (if available) commercial EDR; metrics documented
11-12	Feature Freeze	No new features; bug fixes and stabilization only
13-14	Documentation	Complete technical documentation, user guide, and architecture diagrams
15-16	Final Showcase	Final demonstration; poster; presentation; all deliverables submitted

Technical Requirements

Development Environment

- Language: C (x64 only to reduce complexity)
- Compiler: Visual Studio 2022 with Windows SDK
- Testing: Isolated Windows 10/11 VMs with snapshots — NEVER test on host machines
- Version Control: GitHub repository with branch protection and code review requirements

Suggested Tools

- PE Analysis: PE-bear, CFF Explorer, PEstudio
- Disassembly: IDA Free or Ghidra
- Debugging: x64dbg, WinDbg, Process Hacker
- String Analysis: FLOSS (Mandiant)
- Syscall Reference: System call tables, ntdll disassembly
- Hash Lookup: HashDB (OALabs)

Validation Metrics

Success is measured by functional effectiveness and learning demonstrated, not solely by detection rates. Document the following:

- Entropy: Per-section values before/after (target >7.0 for encrypted sections)
- String Visibility: FLOSS extraction count reduction (target 80-95% reduction)
- Import Table: IAT entry count reduction (target <10 visible imports)
- Injection Success: Payload executes correctly in target process
- Evasion Testing: Document behavior against Windows Defender and available EDR platforms

Testing Environment

Testing against commercial EDR platforms (such as CrowdStrike, SentinelOne, etc.) will require coordination with ProDefense to provide appropriate testing environments. This may include isolated lab environments or access to EDR platforms for controlled testing.

Alternative testing approaches using open-source solutions or Windows Defender may be used if commercial EDR access cannot be arranged. The team should begin testing with Windows Defender and progress to commercial solutions as access becomes available.

Learning Resources

Packer/Crypter References

- WireDiver PE Packer Tutorial (wirediver.com) — Complete 5-part implementation guide
- czs108/Windows-PE-Packer (GitHub) — Beginner-friendly reference implementation
- Practical Malware Analysis, Chapter 18 — Packers and unpacking methodology
- Microsoft PE Format Specification — Authoritative header structure reference

Loader/Injection References

- ired.team notes — Hands-on code samples for process injection and evasion
- Elastic "Ten Process Injection Techniques" — Survey with MITRE ATT&CK mapping
- SysWhispers2/SysWhispers3 (GitHub) — Syscall stub generation reference
- modexp blog — Detailed Windows internals and injection techniques

Syscall and Evasion References

- Hell's Gate / Halo's Gate papers — Dynamic syscall number resolution
- MDSec blog posts — AMSI/ETW bypass techniques
- "Malware Obfuscation Techniques: A Brief Survey" (You & Yim, 2010) — Technique taxonomy

Communication and Expectations

Meeting Schedule

- Sprint Retrospectives: Per CIDSE capstone schedule (sponsors review drafts before meetings)
- Weekly Check-ins: 15-minute standup format — "What's blocking you?" focus
- Sponsor Evaluations: Formal reviews every 8 weeks via CIDSE surveys

Team Expectations

- Time Commitment: 10-12 hours per week per team member
- Code Review: No merge without review from a different team member
- Documentation: Exploration reports required for each technique investigated

- Communication: Promptly flag blockers — don't wait until sprint retrospectives

Success Criteria

This project prioritizes learning over production-ready tools. Success is defined as:

Baseline (Required for Project Completion)

- Working PE parser that extracts and displays headers/sections
- XOR or RC4 encryption with self-decrypting stub
- At least 2 additional obfuscation techniques fully implemented
- At least 1 process injection technique working (DLL injection or process hollowing)
- Basic AMSI bypass implemented
- Documented test results with quantitative metrics

Expected (Team On Track)

- String encryption reducing FLOSS extraction by >80%
- API obfuscation hiding most imports
- Metadata manipulation (timestamps, Rich header, sections)
- Multiple injection techniques available
- Direct syscalls implemented for key NT functions
- ETW bypass functional
- Comprehensive exploration reports for techniques investigated

Stretch (Ahead of Schedule)

- Indirect syscalls with dynamic syscall number resolution
- ntdll unhooking to restore clean copies
- Control flow flattening on demonstration function
- Module stomping injection
- GUI wrapper for tools